

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Численное интегрирование. Составные формулы прямоугольников,
трапеций, Симпсона.

Студентка гр. 4342

Волкова В.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Используя составные квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значение заданного определенного интеграла. С помощью правила Рунге найти минимальное число разбиений n , при котором оценка погрешности не превышает заданную точность ε , и сравнить применяемые формулы между собой.

Основные теоретические положения.

Численное интегрирование применяется тогда, когда первообразная подынтегральной функции неизвестна или ее неудобно использовать для точного вычисления интеграла. В этом случае интеграл заменяется конечной суммой значений функции в выбранных узлах.

Для повышения точности используются составные формулы. Отрезок $[a, b]$ разбивается на n равных частей с шагом $h = (b - a) / n$. Чем меньше шаг h , тем больше узлов используется и тем точнее получается приближение интеграла.

В работе рассматриваются три составные формулы: формула средних прямоугольников, формула трапеций и формула Симпсона.

$$I = \int_a^b f(x) dx$$

Составная формула средних прямоугольников имеет вид:

$$I \approx h \cdot \sum_{i=0}^{n-1} f(x_i + h/2).$$

В этой формуле на каждом маленьком отрезке значение функции берется в середине, поэтому метод часто дает более точный результат, чем формула левых или правых прямоугольников.

Составная формула трапеций заменяет график функции на каждом отрезке прямой линией, то есть площадь под кривой приближенно заменяется площадью трапеции:

$$I \approx h \cdot (f(x_0)/2 + f(x_1) + \dots + f(x_{n-1}) + f(x_n)/2).$$

Составная формула Симпсона использует параболическую аппроксимацию функции на парах соседних отрезков. Для ее применения число разбиений n должно быть четным:

$$I \approx h/3 \cdot (f(x_0) + 4f(x_1) + 2f(x_2) + \dots + 4f(x_{n-1}) + f(x_n)).$$

Для практической оценки погрешности квадратурной можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$$I_{h/2} + |I_{h/2} - I_h| / 3 \text{ - для формулы прямоугольников;}$$

$$I_{h/2} - |I_{h/2} - I_h| / 3 \text{ - для формулы трапеций;}$$

$$I_{h/2} - |I_{h/2} - I_h| / 15 \text{ - для формулы Симпсона.}$$

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину $|I_{h/2} - I_h| / 3$, а для формулы Симпсона $|I_{h/2} - I_h| / 15$.

Итерационный процесс в программе устроен так: значение n удваивается до тех пор, пока оценка R не станет меньше заданной точности ε . После этого найденное значение n и приближенное значение интеграла выводятся как результат.

Выполнение работы.

Для вычислительного эксперимента выбран интеграл

$$I = \int_0^1 \sin(x^4 + 2x^3 + x^2) dx.$$

Подынтегральная функция $f(x) = \sin(x^4 + 2x^3 + x^2)$ непрерывна и достаточно гладкая на отрезке $[0, 1]$, поэтому для нее можно применять все три рассматриваемые составные формулы. Контрольное значение интеграла, полученное численным расчетом с высокой точностью, равно

$$I \approx 0.3629205713.$$

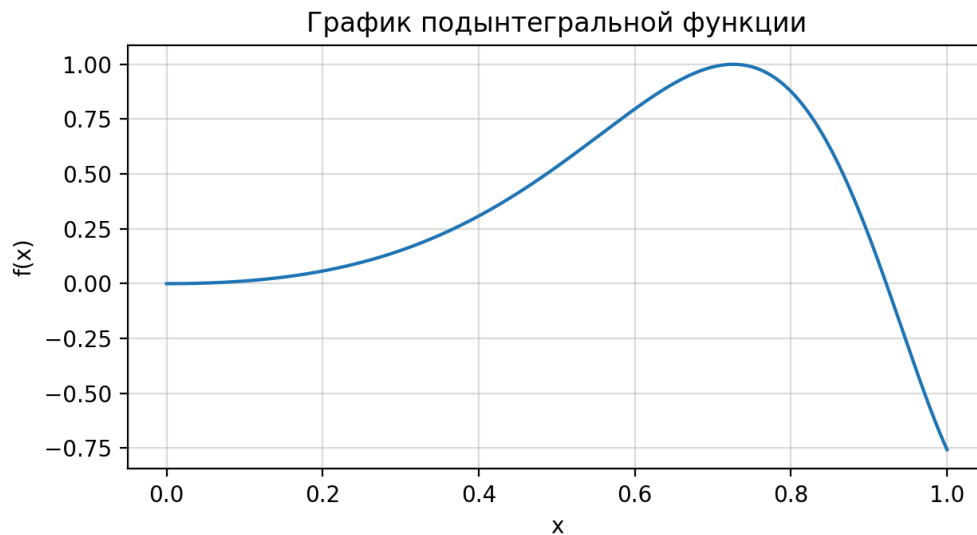


Рисунок 1 — график подынтегральной функции $f(x)$ на отрезке $[0, 1]$

На графике видно, что функция на выбранном отрезке не имеет разрывов и резких скачков. Это объясняет устойчивое уменьшение погрешности при уменьшении шага интегрирования.

В программе были реализованы отдельные функции для вычисления интеграла по каждой формуле. Для каждой точности $\varepsilon = 0.01, 0.001, 0.0001$ выполнялось последовательное удвоение n . В таблице n и h указаны для более мелкой итоговой сетки. Столбец I_h соответствует предыдущей, более грубой сетке, а столбец $I_{\square/2}$ - итоговой сетке, по которой берется уточненное значение интеграла.

Таблица 1 — результаты вычисления интеграла по правилу Рунге

Формула	ε	n	h	I_h	$I_{\square/2}$	R	$I_{\text{уточн.}}$
Прямоугольников	10^{-2}	8	0.125000	0.3938757 150	0.3686070 599	8.423e-03	0.3601841 749
Прямоугольников	10^{-3}	32	0.031250	0.3642313 661	0.3632418 335	3.298e-04	0.3629119 893
Прямоугольников	10^{-4}	64	0.015625	0.3632418 335	0.3630004 925	8.045e-05	0.3629200 456
Трапеций	10^{-2}	16	0.062500	0.3520496 017	0.3603283 308	2.760e-03	0.3630879 072

Формула	ε	n	h	I_h	$I_{\square/2}$	R	$I_{\text{уточн.}}$
Трапечий	10^{-3}	32	0.031250	0.3603283 308	0.3622798 485	6.505e-04	0.3629303 543
Трапечий	10^{-4}	128	0.007812	0.3627608 410	0.3628806 668	3.994e-05	0.3629206 087
Симпсона	10^{-2}	8	0.125000	0.3878144 136	0.3659916 395	1.455e-03	0.3645367 879
Симпсона	10^{-3}	16	0.062500	0.3659916 395	0.3630879 072	1.936e-04	0.3628943 250
Симпсона	10^{-4}	32	0.031250	0.3630879 072	0.3629303 543	1.050e-05	0.3629198 508

Из таблицы видно, что при уменьшении ε требуется увеличивать число разбиений. Для точности 10^{-4} формуле прямоугольников потребовалось $n = 64$, формуле трапечий - $n = 128$, а формуле Симпсона - $n = 32$. Следовательно, на данном примере формула Симпсона оказалась наиболее эффективной по числу разбиений.

Формула трапечий в данном расчете требует больше разбиений, чем формула средних прямоугольников. Это связано с тем, что формула прямоугольников использует значение функции в середине каждого отрезка, и для данной гладкой функции это дает хорошее взаимное компенсирование ошибок.

Формула Симпсона имеет более высокий порядок точности. Поэтому при достаточно строгих требованиях к точности ее погрешность убывает быстрее, чем у формул прямоугольников и трапечий.

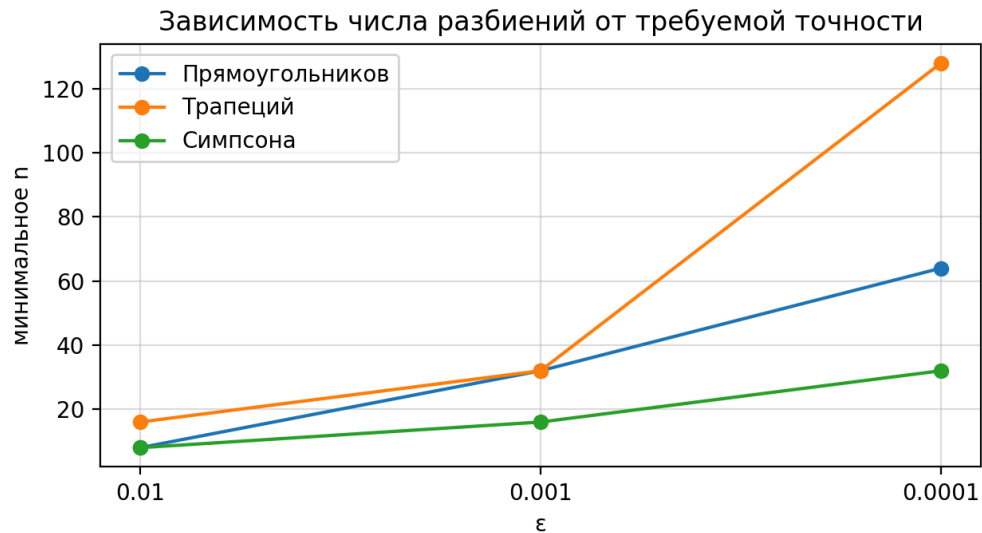


Рисунок 2 — зависимость минимального числа разбиений n от требуемой точности ϵ

По рисунку 2 видно, что при ужесточении точности наиболее резко растет число разбиений у формулы трапеций. У формулы Симпсона рост n меньше, так как ее порядок точности выше.

Описание функций программы.

Функция $F(x)$ вычисляет значение подынтегральной функции $\sin(x^4 + 2x^3 + x^2)$. Она используется всеми тремя квадратурными формулами.

Функция `rectangleFormula(a, b, n)` реализует составную формулу средних прямоугольников. В цикле перебираются все n промежутков, для каждого промежутка берется его середина, после чего сумма умножается на шаг h .

Функция `trapezoidFormula(a, b, n)` реализует составную формулу трапеций. Значения функции на концах отрезка учитываются с коэффициентом $1/2$, а внутренние значения - с коэффициентом 1 .

Функция `simpsonFormula(a, b, n)` реализует составную формулу Симпсона. В ней значения функции в нечетных внутренних узлах умножаются на 4 , а в четных

внутренних узлах - на 2. Если n нечетное, программа сообщает об ошибке, потому что формула Симпсона требует четного числа разбиений.

Функция `solveByRunge(method, order, a, b, eps)` выполняет основной вычислительный процесс. Она считает интеграл на сетках с n и $2n$ разбиениями, оценивает погрешность по правилу Рунге и удваивает n до тех пор, пока погрешность не станет меньше ϵ .

Функция `printResult(name, result)` отвечает за аккуратный вывод результата: название формулы, требуемую точность, найденное значение n , шаг h , оценку погрешности и уточненное значение интеграла.

Выводы.

В ходе лабораторной работы были реализованы составные формулы прямоугольников, трапеций и Симпсона для приближенного вычисления определенного интеграла. Для контроля точности использовалось правило Рунге, позволяющее оценить погрешность по двум расчетам с шагами h и $h/2$.

Для интеграла $\int_0^1 \sin(x^4 + 2x^3 + x^2) dx$ все три формулы дали результаты, сходящиеся к одному и тому же значению $I \approx 0.3629205713$. При уменьшении требуемой погрешности ϵ число разбиений n увеличивалось, что соответствует теории численного интегрирования.

Наиболее эффективной оказалась формула Симпсона: для $\epsilon = 10^{-4}$ ей потребовалось только $n = 32$ разбиения. Формула прямоугольников дала приемлемый результат при $n = 64$, а формула трапеций потребовала $n = 128$. Это подтверждает, что формула Симпсона имеет более высокую скорость уточнения результата.

Таким образом, для гладких функций формула Симпсона обычно является наиболее выгодной, так как позволяет получить заданную точность при меньшем числе разбиений. Формулы прямоугольников и трапеций проще по вычислительной схеме, но для высокой точности требуют более мелкой сетки.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл integral.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <stdexcept>
#include <string>

using namespace std;

double F(double x)
{
    return sin(pow(x, 4) + 2.0 * pow(x, 3) + x * x);
}

double rectangleFormula(double a, double b, int n)
{
    double h = (b - a) / n;
    double sum = 0.0;

    for (int i = 0; i < n; i++)
    {
        double x = a + (i + 0.5) * h;
        sum += F(x);
    }

    return h * sum;
}

double trapezoidFormula(double a, double b, int n)
{
    double h = (b - a) / n;
    double sum = 0.5 * (F(a) + F(b));

    for (int i = 1; i < n; i++)
    {
        double x = a + i * h;
        sum += F(x);
    }

    return h * sum;
}

double simpsonFormula(double a, double b, int n)
{
    double h = (b - a) / n;
    double sum = F(a) + F(b);

    for (int i = 1; i < n; i++)
    {
        double x = a + i * h;

        if (i % 2 == 0)
            sum += 2.0 * F(x);
        else
            sum += 4.0 * F(x);
    }
}
```

```

    }

    return h * sum / 3.0;
}

struct Result
{
    int n;
    double h;
    double valueH;
    double valueHalfH;
    double rungeCorrection;
    double integralValue;
};

typedef double (*Method)(double, double, int);

Result solveByRunge(Method method, double divider, int sign, double a, double b,
double eps)
{
    int n = 2;

    while (true)
    {
        double valueH = method(a, b, n);
        double valueHalfH = method(a, b, 2 * n);

        double rungeCorrection = fabs(valueHalfH - valueH) / divider;

        double integralValue = valueHalfH + sign * rungeCorrection;

        if (rungeCorrection <= eps)
        {
            Result result;
            result.n = 2 * n;
            result.h = (b - a) / result.n;
            result.valueH = valueH;
            result.valueHalfH = valueHalfH;
            result.rungeCorrection = rungeCorrection;
            result.integralValue = integralValue;
            return result;
        }

        n *= 2;
    }
}

void printResult(const string& name, double eps, const Result& result)
{
    cout << name << endl;
    cout << "eps = " << eps << endl;
    cout << "n = " << result.n << endl;
    cout << "h = " << result.h << endl;
    cout << "I_h = " << result.valueH << endl;
    cout << "I_h/2 = " << result.valueHalfH << endl;
    cout << "Runge correction = " << result.rungeCorrection << endl;
    cout << "Integral = " << result.integralValue << endl;
    cout << endl;
}

int main()
{
    const double a = 0.0;

```

```

const double b = 1.0;

double eps;

cout << "Enter eps: ";
cin >> eps;

if (eps <= 0.0)
{
    cout << "Error: eps must be positive" << endl;
    return 1;
}

cout << fixed << setprecision(10);

cout << "=====" << endl;
cout << "eps = " << eps << endl << endl;

Result rect = solveByRunge(rectangleFormula, 3.0, 1, a, b, eps);
Result trap = solveByRunge(trapezoidFormula, 3.0, -1, a, b, eps);
Result simp = solveByRunge(simpsonFormula, 15.0, -1, a, b, eps);

printResult("Rectangle formula", eps, rect);
printResult("Trapezoid formula", eps, trap);
printResult("Simpson formula", eps, simp);

return 0;
}

```